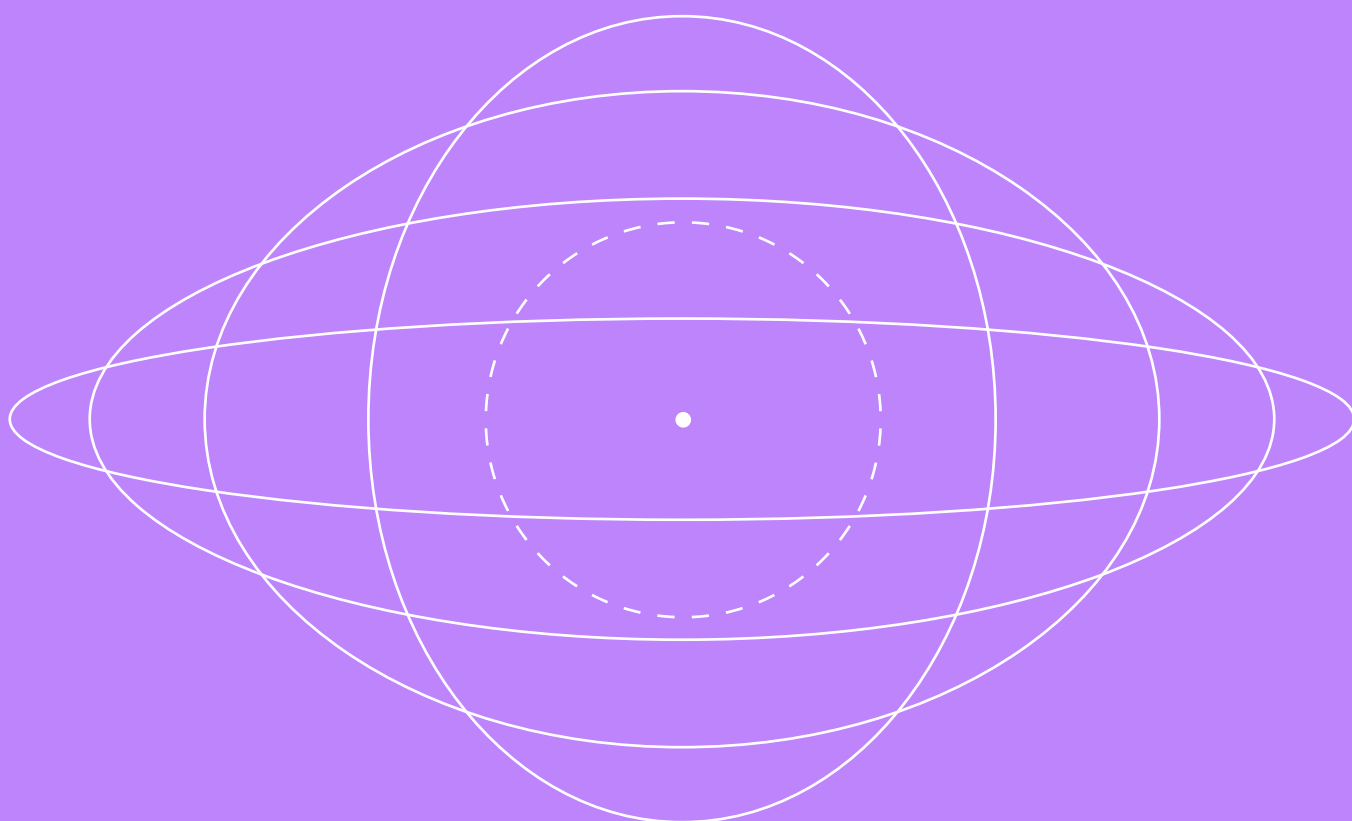


Буферный кеш и журнал предзаписи



SQL И БАЗЫ ДАННЫХ (ДЛЯ РАЗРАБОТЧИКОВ)

НЕДЕЛЯ 14

Рассмотрим архитектуру буферного кеша (Buffer Pool) и журнал предзаписи (Write-Ahead Log), а также их совместную роль в обеспечении транзакционной целостности (ACID) и высокой производительности. Структура охватывает иерархию памяти, внутреннее устройство буферного пула, алгоритмы вытеснения страниц, теорему ACID-I/O (политики Steal/No-Force), протокол WAL и механизмы восстановления после сбоев (Crash Recovery), включая алгоритм ARIES.

Проблема архитектуры СУБД заключается в значительном разрыве задержек между оперативной памятью и постоянными накопителями. Время доступа к RAM измеряется наносекундами (~100 нс), однако данные волатильны и теряются при сбое питания. Доступ к твердотельным накопителям (SSD/NVMe) занимает микросекунды (~10–100 мкс), что на два-три порядка медленнее RAM, а к магнитным дискам (HDD) — миллисекунды, что медленнее на пять порядков. Дизайн СУБД строится на компромиссе: обработка данных должна происходить на скоростях оперативной памяти, при этом система обязана обеспечивать персистентность уровня жёсткого диска.

СУБД корпоративного класса минимизируют использование кеша операционной системы, предпочитая прямой ввод-вывод (например, флаг O_DIRECT в Linux). Этому есть три архитектурные причины. Во-первых, проблема двойной буферизации: данные хранятся и в памяти ОС, и в памяти СУБД, что приводит к неэффективному расходованию RAM. Во-вторых, ОС применяет универсальные алгоритмы вытеснения страниц, так как не понимает семантики запросов базы данных. Операционная система может вытеснить критически важный корневой узел B-дерева, сохранив транзитные данные от последовательного сканирования таблицы. В-третьих, кеш ОС может нарушить порядок записи данных и логов на диск, что делает невозможным консистентное восстановление системы после сбоя.

ОПРЕДЕЛЕНИЕ

Буферный пул (Buffer Pool) — это область оперативной памяти, выделенная СУБД для кэширования данных. Ключевым является разделение понятий физического и логического хранения:

- **page (страница)** — блок данных фиксированного размера (обычно 8 KB или 16 KB), являющийся минимальной единицей чтения/записи на диск;
- **frame (кадр)** — предварительно выделенный слот в оперативной памяти (внутри Buffer Pool), размер которого строго равен размеру одной страницы. Кадр выступает контейнером для размещения страницы во время её обработки.

Для маршрутизации запросов к данным в памяти используется таблица страниц (Page Table). Это глобальная хеш-таблица, обеспечивающая мэппинг идентификатора дисковой страницы (Page_ID) на номер кадра в оперативной памяти (Frame_ID). При запросе данных возможны два сценария. Попадание в кеш: страница найдена в таблице, СУБД напрямую обращается к кадру. Пропах кеша: страница отсутствует в памяти; СУБД выделяет свободный кадр, иницирует синхронное чтение страницы с диска и обновляет Page Table.

Для обеспечения корректной работы высококонкурентной среды необходимо строго разграничивать механизмы синхронизации:

- **locks (блокировки)** — логические примитивы, защищающие содержимое БД (строки, таблицы) от конфликтов между транзакциями. Они удерживаются длительное время (до завершения транзакции) и поддерживают механизмы обнаружения взаимных блокировок (Deadlock Detection);
- **latches (защёлки)** — физические примитивы синхронизации (мьютексы/спинлоки), защищающие внутренние структуры памяти СУБД (например, Page Table или отдельные кадры) от одновременного изменения потоками ОС. Они удерживаются микросекунды и не имеют встроенной защиты от дедлоков.

Каждый кадр в буферном пуле обладает метаданными, описывающими его текущий статус:

- **free (свободен)** — кадр не содержит полезных данных и готов к записи новой страницы;
- **pinned (закреплён)** — страница в кадре в данный момент читается или модифицируется исполнителем запроса. Использование отслеживается счётчиком Pin Count. Кадр не подлежит вытеснению, пока Pin Count > 0;
- **dirty (грязный)** — страница была изменена в оперативной памяти, и её содержимое рассинхронизировано с копией на диске. Перед освобождением такого кадра система обязана сбросить (flush) измененную страницу на накопитель.

При исчерпании пула свободных кадров СУБД должна инициировать процедуру вытеснения (Page Replacement). Классическим подходом является алгоритм Least Recently Used (LRU) — вытеснение страницы, к которой дольше всего не было обращений. Технически это реализуется через двусвязный список: при каждом обращении страница перемещается в «голову» списка, а жертва для вытеснения всегда выбирается из «хвоста». Алгоритм опирается на принцип временной локальности (temporal locality), предполагая, что недавно использованные данные понадобятся снова.

Несмотря на эффективность в общих вычислительных задачах, наивный LRU имеет критическую уязвимость в контексте баз данных, известную как Sequential Flooding (последовательное затопление). Проблема возникает при выполнении полного сканирования таблицы (Full Table Scan), размер которой превышает объём буферного пула. Алгоритм LRU начинает последовательно загружать новые страницы, вытесняя в «хвост» и удаляя из памяти исторически часто используемые данные (индексы, «горячие» строки). В результате буферный пул полностью заполняется страницами сканируемой таблицы, которые были прочитаны однократно и более не требуются, что приводит к резкой деградации производительности всей СУБД до скорости физического чтения с диска.

Для минимизации накладных расходов процессора и предотвращения вымывания кеша применяется алгоритм Clock (часовой алгоритм). В нём кадры организованы в виде кольцевого буфера, а каждому кадру назначается бит обращения (Reference Bit). При обращении к странице бит устанавливается в единицу. Фоновый процесс циклично обходит буфер: если бит равен единице, он сбрасывается в ноль (предоставление «второго шанса»), если равен нулю — страница вытесняется. Это обеспечивает эффективную аппроксимацию LRU с минимальными затратами ресурсов.

Для надёжной защиты от последовательного сканирования (Sequential Flooding) базовый алгоритм Clock модифицируется. Современные СУБД используют подходы на базе LRU-K и 2Q. Суть заключается в логическом разделении буфера на транзитную зону («карантин») и зону «горячих» данных. Страница перемещается в защищённую «горячую» зону только после повторного обращения. Транзитные данные от сканирования таблиц вытесняются из карантинной зоны, не затрагивая полезные индексы.

Управление буферным пулом описывается через матрицу политик взаимодействия оперативной памяти и диска, состоящую из двух осей. Первая ось (вытеснение) определяет, разрешено ли СУБД сохранять на диск изменённые страницы транзакции, которая ещё не зафиксирована (COMMIT). Вторая ось (фиксация) определяет, обязана ли система синхронно сбрасывать все изменения транзакции на диск строго в момент фиксации.

Политика No-Steal запрещает вытеснение «грязных» страниц до фиксации транзакции. Это гарантирует отсутствие неконсистентных данных на диске при отмене транзакции, но жёстко ограничивает максимальный объём транзакции размером доступной оперативной памяти. Политика Steal разрешает вытеснение страниц незафиксированных транзакций для освобождения памяти. Это снимает ограничения на размер транзакции, но требует наличия механизма отмены (UNDO) для восстановления согласованного состояния при сбое БД или команде ROLLBACK.

Политика Force требует синхронной записи всех «грязных» страниц транзакции на диск в момент COMMIT. Это обеспечивает надёжность (Durability), но приводит к значительному падению производительности из-за множества мелких случайных (random) операций ввода-вывода.

Политика No-Force завершает фиксацию транзакции исключительно в памяти; данные сбрасываются на диск асинхронно. Это обеспечивает максимальную пропускную способность, но в случае аппаратного сбоя требует механизма повтора (REDO) для восстановления подтверждённых транзакций.

Промышленные реляционные движки используют комбинацию политик Steal и No-Force, поскольку она обеспечивает поддержку транзакций неограниченного размера и максимальную производительность ввода-вывода. Сама по себе эта связка нарушает гарантии атомарности и долговечности, что делает абсолютно обязательным использование внешнего независимого журнала для обеспечения недостающих функций UNDO и REDO.

Для обеспечения надёжности в архитектуре Steal/No-Force применяется журнал предзаписи (WAL) — отдельный файл (или набор файлов) для строго последовательной (Append-Only) фиксации всех изменений. Фундаментальное правило WAL: запись об изменении в журнале обязана быть физически зафиксирована на энергонезависимом носителе до того, как соответствующая «грязная» страница данных будет вытеснена из оперативной памяти на диск.

Синхронизация состояния памяти и диска опирается на Log Sequence Number (LSN) — глобальный, монотонно возрастающий 64-битный счётчик. Каждая новая запись в WAL получает уникальный LSN, который физически соответствует смещению записи в байтах от начала журнала.

LSN выполняет роль логического времени внутри базы данных.

Работа протокола WAL формализуется через две переменные. Переменная PageLSN хранится в заголовке каждой страницы данных и обновляется при каждом её изменении (фиксируется LSN последней операции). Переменная FlushedLSN является глобальной в памяти и указывает на максимальный LSN, гарантированно записанный в WAL на диске. Буферный менеджер имеет право вытеснить «грязную» страницу на диск исключительно при выполнении условия: $PageLSN \leq FlushedLSN$. Если условие не выполнено, инициируется принудительный сброс «хвоста» WAL на накопитель.

Существует три подхода к формату записей в WAL. Физическое логирование (сохранение полных копий байтов) приводит к избыточному размеру журнала. Логическое логирование (сохранение SQL-инструкций) компактно, но не детерминировано при восстановлении.

Стандартом является физиологическое логирование (physiological logging), которое фиксирует физические изменения на уровне конкретной логической структуры (например, операция вставки записи в определённый слот страницы), обеспечивая баланс между объёмом журнала и строгой детерминированностью восстановления.

Реализация СУБД неизбежно сталкивается с физическими ограничениями аппаратного обеспечения при записи данных. СУБД оперирует страницами данных размером 8 KB (PostgreSQL) или 16 KB (InnoDB). Однако современные накопители и операционные системы аппаратно гарантируют атомарность записи только на уровне сектора (512 байт) или блока (4 KB). Если во время записи полной страницы на диск происходит сбой питания, возникает проблема частичной записи (Torn Page): одна половина страницы содержит новые данные, а другая — старые. Контрольная сумма такой страницы нарушается, и данные повреждаются, поскольку алгоритмы восстановления не могут корректно применить изменения из WAL к физически разрушенной базовой структуре.

Для предотвращения потери данных из-за частичной записи применяются два основных архитектурных подхода. Первый — запись полных страниц, используемая в PostgreSQL. При первом изменении страницы после контрольной точки в WAL записывается не дельта изменений, а полная физическая копия страницы. При сбое повреждённая страница на диске заменяется своей эталонной копией из журнала. Второй подход — буфер двойной записи, характерный для InnoDB. Перед сохранением в основные файлы данных изменённые страницы строго последовательно сбрасываются в выделенную зону на диске. В случае падения во время основной записи неповреждённая копия извлекается из этого буфера.

В штатном режиме работы СУБД журнал предзаписи (WAL) функционирует исключительно на запись. Он не используется для обслуживания пользовательских запросов на чтение или обновление данных. WAL читается системой только в одном случае — при перезапуске сервера после аварийного отключения (Crash Recovery). Главная задача процесса восстановления — устранить физическую неконсистентность данных на диске, математически точно воссоздав состояние оперативной памяти на момент, предшествующий сбою.

Журнал предзаписи не может расти бесконечно, а время восстановления базы данных (RTO) должно быть предсказуемым. Для решения этой задачи используется механизм контрольных точек (Checkpoints). Это фоновый процесс, который принудительно сбрасывает все текущие «грязные» страницы из оперативной памяти на диск и фиксирует это событие в WAL. Контрольная точка устанавливает границу (Redo LSN), гарантируя, что все изменения до этой отметки физически зафиксированы на постоянном носителе. Старые сегменты журнала, предшествующие этой точке, больше не требуются для восстановления при сбое и могут быть удалены или перезаписаны.

В основе механизмов восстановления большинства современных СУБД лежит алгоритм ARIES (Algorithm for Recovery and Isolation Exploiting Semantics). Архитектура алгоритма базируется на принципе «повторения истории» (Repeating History): система сначала безусловно воссоздаёт картину данных на момент аварии, а лишь затем корректно отменяет незавершённые операции. Процесс восстановления строго разделён на три последовательных этапа: фазу анализа, фазу повтора и фазу отмены.

На первом этапе восстановления СУБД сканирует WAL в прямом направлении от последней успешной контрольной точки до конца журнала. Цель фазы — воссоздать состояние внутренних структур данных на момент сбоя. Формируется «Таблица транзакций» для выявления незафиксированных транзакций и «Таблица грязных страниц» для определения точного LSN, с которого необходимо начать накат данных, чтобы избежать избыточного чтения диска.

Второй этап реализует принцип «повторения истории». СУБД читает журнал вперёд, начиная с вычисленного на предыдущем шаге Redo LSN, и безусловно применяет к страницам данных все зафиксированные изменения. Накат операций происходит с соблюдением правила идемпотентности: изменение применяется к странице только в том случае, если LSN записи в логе строго больше LSN текущей дисковой страницы ($\text{LogRecordLSN} > \text{PageLSN}$). В результате оперативная память возвращается в точное состояние за микросекунду до отключения питания, включая «мусор» от незавершённых транзакций.

Третий этап обеспечивает гарантию атомарности. СУБД читает журнал в обратном направлении (от конца к началу) и выполняет логический откат операций для всех незавершённых транзакций, выявленных на фазе анализа. Ключевой особенностью протокола является то, что сами операции отмены также протоколируются в WAL в виде компенсационных записей (Compensation Log Records, CLR). Если сервер повторно упадёт во время фазы отмены, при следующем запуске алгоритм продолжит откат с прерванного места, полностью исключая риск двойной отмены изменений.

Архитектура подсистемы ввода-вывода базируется на синергии двух компонентов. Менеджер буферного пула обеспечивает производительность системы, позволяя обрабатывать данные со скоростью оперативной памяти и маскируя задержки дисковых операций через алгоритмы вытеснения и асинхронную запись. Журнал предзаписи (WAL) обеспечивает транзакционную надёжность (ACID), позволяя безопасно использовать агрессивные политики управления кешем (Steal/No-Force). Жёсткая синхронизация состояний летучей памяти и постоянного накопителя обеспечивается механизмом Log Sequence Number (LSN), который гарантирует целостность данных в любых сценариях эксплуатации.